Introduction

Along these tutorials, we will see how **SCOPE** interacts with the different parts of the code to handle the execution of computational workflows.

These are the topics covered in each tutorial:

1. The **System** class and its sources: the **Specie**, **Cell** and **Atom** classes
2. The Computational workflow: **Branch**, **Workflow**, **Job**, and **Computation** classes
3. The **State** class
4. The **Data**, **Collection** and **VNM** classes
5. The **Input_data** class, and **scope input files**
6. Running **SCOPE** - Part 1: File Structure
7. Running **SCOPE** - Part 2: Execution
8. Running **SCOPE** - Part 3: Detailed Actions
9. The Azo add-on: Subclasses, Analysis and Creation of Azo Systems

Tutorial 6: Running SCOPE, Part 1

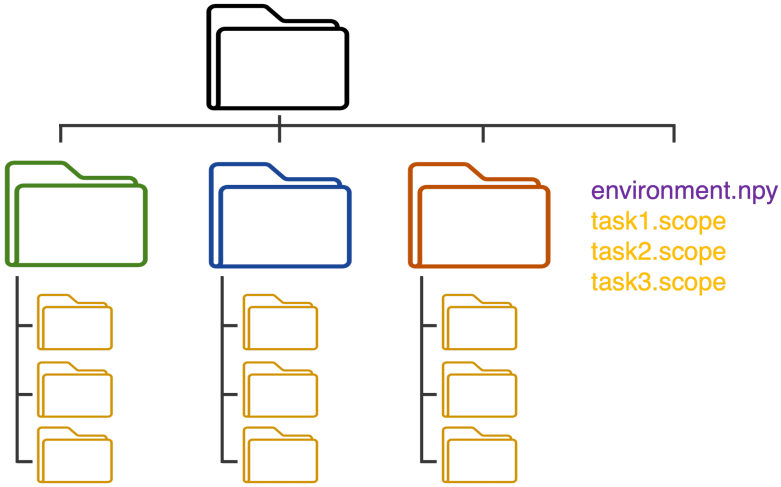
Expected File Structure

SCOPE interacts with five types of files:

- Binary files containing SYSTEM-class objects, discussed in **Tutorial 1** (**SYSTEM**)
- Plain text (or binary) files that are used to create SOURCES for systems, as discussed in **Tutorial 1** (**SOURCE**)
- Plain text input, output and submission files associated with the computation (**COMPUTATIONS**)
- Plain text SCOPE input FILES describing tasks, discussed in **Tutorial 5** (**INPUT**)
- Binary files of ENVIRONMENT-class objects, as we will discuss here (**ENVIRONMENT**)

The first three accumulate very rapidly, so they are stored in SYSTEM-specific folders, whereas **INPUT** and **ENVIRONMENT** files are specific of each project

Loading [MathJax]/extensions/Safe.js



Scheme 1: Expected folder structure for projects in SCOPE. Environment and Input files inside the project folder (in black). Each system must have dedicated folders (gold) inside the main Systems (green), Sources (blue), and Computations (red) folders. Other folder structures can be arranged, but won't be discussed here.

SCOPE minimizes the interaction between the user and the projects files, thanks to the **ENVIRONMENT** class, which stores all relevant paths in a given computer. This also simplifies sharing data between the local computer (which is where you would typically analyse the results) and the computational clusters (where you run the computations).

The key path variables in the Environment-class object are:

- sources_path
- systems_path
- computations_path

Let's say you have a local computer ("LOCAL") and a computational HPC cluster ("HPC1") at your disposal. To move files between these computers, you just need to:

- Create an environment in each computer. That is, run "scope config" in LOCAL and HPC1
- Run you SCOPE tasks in HPC1
- Syncronize the LOCAL folders (at least SYSTEMS and COMPUTATIONS) with the files in HPC1 (for instance, with rsync)
- You can freely interact with the System files you downloaded, without having to modify the paths. Whenever SCOPE requires the actual path of a source, system, or computation file, it will reconstruct it from the evironment.

Dos and don'ts

Loading [MathJax]/extensions/Safe.js

- You can freely move the main folder of the project (black folder in scheme 1), as long as you update all environment paths running env.set_paths() on the existing environment. Of course you can also create a new environment with the new paths

- Systems:

- Can be deleted harmlessly as long as COMPUTATION files remain in place (i.e. same path, same filename): if inputs and computation files haven't changed, re-running Scope will re-generate the SYSTEM.
- You can move the entire SYSTEMS folder, preserving the file structure inside (subfolders), and updating the env.systems_path attribute of the existing environment
- Ideally, you should update all systems (one by one) with the SYSTEM-class function: sys.set_paths_from_environment(env), where env and sys are the environment and system objects, respectively.

- Sources:

- Once Sources are imported to the SYSTEM, the source files (e.g. .xyz) can be deleted, at the risk of being unable to re-generate the SYSTEM if something happens.

- Computation files:

- Do not remove those files unless you're really sure what you're doing, even if the results have been registered already. They're your backup to reconstruct SYSTEMS if necessary.
- If you move the Computations folder, preserve the file structure inside, and change the env.computations_path in the environment
  Otherwise, SCOPE will re-submit these very same computations, wasting computational resources (!)

- Job Files:

- You can move or delete these files harmlessly
- You can modify the *&environment* and *&options* sections anytime during the execution of tasks (Cases 1-4 below)
- Avoid modifying the *&job_data* and *&qc_data* sections as much as possible.

Example:

Loading [MathJax]/extensions/Safe.js

Imagine you want to run 2 tasks (*task1*, *task2*), with *task2* requiring *task1* to finish. To finish both tasks, you'll need to execute the following command several times (at least 3, see below) in a computation cluster (i.e. a cluster with a job manager, SLURM or SGE) with a configured environment:

```
scope run -n path_to_environment_file -s system_name -i task1.scope
task2.scope -e
```

Environment:

To configure an environment you can run the script "**scope config**" in the computation cluster, and follow instructions.

```
scope config -h
```

You can also configure a scope environment in your local computer, to analyze the data. The single functionality of SCOPE that is inactive in a local computer is the submission of jobs. If those have already been submitted, and the files are properly stored in the env.computations_path of the environment in your local computer, you can run the "**scope run**" command above in your local computer to register the data. This can be useful if you regenerate a system and want to re-register the results.

Execution:

(1) The first time you execute the **scope run** command above, SCOPE will submit the computation(s) associated with task1 to the cluster. The System file will be updated and saved
(2) If you execute once again while task1's computations are still running, nothing will happen. The System file won't be updated
(3) If you execute once task1's computations finish, Scope will register task1, and prepare and submit task2. The System file will be updated
(4) If you execute once task2's computations finish, Scope will skip task1 (it is already registered), register task2, and the workflow will be over. The System file will be updated, and you can then analyse the results.

Add new Tasks:

Once tasks 1 and 2 have finished, you can add additional tasks (task3):

```
scope run -n path_to_environment_file -s system_name -i task1.scope
task2.scope task3.scope -e
```

or simply:

```
scope run -n path_to_environment_file -s system_name -i task3.scope
-e
```

If tasks 1 and 2 are already registered, then the two options above should be equivalent

Loading [MathJax]/extensions/Safe.js

task1.scope is only a placeholder. You can give files any name

---

Real Case

In tutorials 7-8, you will experiment with this task-submission system with SCOPE. I expect you to set up a folder structure like the one in Scheme 1.
Otherwise, you can always use Tutorial_7's folder (see below) as the project folder (black folder in Scheme 1). The SOURCES (blue) and SYSTEMS (green) folders already exist, as well as the task files.

"…/Scope_Tutorials/tutorials/Data/7-Tutorial_7"

In [ ]:

```
```

Loading [MathJax]/extensions/Safe.js